

Live Quiz Battle System

Engineering Workflow & Implementation Guide

Full-Stack: Go (3 services) + RabbitMQ + Redis + MongoDB + Flutter

Version 1.0 | Solo Developer One-Day Execution Plan

1. Project Overview

The Live Quiz Battle System is a real-time multiplayer quiz platform where users join matchmaking, are placed in live rooms of 2–10 players, answer timed questions in synchronized rounds, and see their scores update live on a shared leaderboard. Final results are persisted to MongoDB after every match.

The system is built across three Go microservices communicating over gRPC and an async RabbitMQ message bus, with Redis handling all ephemeral game state and a Flutter mobile client consuming gRPC server-streaming for live updates.

1.1 System Components

- Matchmaking & Room Service (Go) — player grouping, room lifecycle, round management, distributed locking
- Quiz Engine Service (Go) — question selection with difficulty balancing, round orchestration, timer management
- Scoring & Ranking Service (Go) — real-time scoring, leaderboard computation, anti-cheat timing validation
- Flutter App — matchmaking UI, live gameplay, leaderboard overlay, results screen
- Redis — all in-flight game state: rooms, leaderboard sorted sets, matchmaking pool, distributed locks
- RabbitMQ — async event bus: match lifecycle events, answer processing, analytics fan-out
- MongoDB — persistent storage: users, question bank, match history

1.2 Technology Stack

Layer	Technology	Rationale
Backend services	Go 1.22+	Low latency, strong concurrency primitives, excellent gRPC support
Service communication	gRPC (protobuf)	Typed contracts, server streaming for live events, efficient binary encoding
Async messaging	RabbitMQ (topic exchange)	Decoupled scoring workers, fan-out for analytics, DLQ for reliability
Live state	Redis 7+	Sorted sets for leaderboard, atomic Lua scripts, pub/sub for fan-out, low-latency reads
Persistent storage	MongoDB 6+	Flexible document model for match history, rich query support for stats
Mobile client	Flutter 3+ / Dart	Single codebase for iOS & Android, strong animation framework

State management	Riverpod	Reactive stream providers map cleanly to gRPC streaming responses
------------------	----------	---

2. End-to-End Game Flow

The following sequence describes a complete match from the moment a player taps Start Battle to the final results screen.

2.1 Matchmaking Phase

1. User taps Start Battle — Flutter sends JoinMatchmaking gRPC call with `userId` and current rating.
2. Matchmaking service adds player to Redis sorted set `matchmaking:pool` with `score = rating`.
3. Background goroutine polls every 500ms. When pool size ≥ 2 players (or any player has waited ≥ 30 s), trigger room creation.
4. Acquire distributed lock `room:lock:{id}` via Redis SETNX with 10s TTL before room creation to prevent race conditions.
5. Atomic MULTI/EXEC: write `room:{id}:players`, `room:{id}:state`, `room:{id}:round` to Redis. Publish `match.created` event to RabbitMQ `sx` exchange.
6. Flutter clients receive match notification via open `SubscribeToMatch` gRPC server stream. Navigate to gameplay screen.

Note: Duplicate matchmaking prevention: before adding to pool, check ZSCORE `matchmaking:pool {userId}`. Return `ALREADY_IN_QUEUE` status if the player is already present.

2.2 Question & Round Phase

7. Quiz Engine consumes `match.created` — selects 5 questions using weighted random pick: 30% easy, 50% medium, 20% hard. No repeat topics in consecutive rounds. Stores ordered list in `room:{id}:questions`.
8. `startRound()` broadcasts `QuestionBroadcast GameEvent` to all clients via `StreamGameEvents`. Payload includes the question text, options array, and an absolute Unix timestamp deadline (not a duration).
9. `time.AfterFunc(15s)` fires on the server. Broadcasts `RoundResult` event with correct answer index regardless of how many answers arrived.

Note: Timer implementation: always broadcast an absolute Unix deadline (e.g. `deadlineUnix: 1712345693`). Flutter computes `deadline - DateTime.now()` locally. This makes the countdown immune to network jitter and ensures both clients display the same number.

2.3 Answer Submission & Scoring Phase

10. Player submits answer — Flutter sends `SubmitAnswer` gRPC call with `roomId`, `round`, `optionIndex`, `clientTimestamp`.
11. Quiz Engine publishes `answer.submitted` event to RabbitMQ `answer-processing-queue`.
12. Scoring Worker checks HEXISTS `room:{id}:answers:{round} {userId}` for idempotency — if already processed, ACK and skip.

13. Score formula: $\text{basePoints} * \text{speedMultiplier}$. Answers in first 5s earn 1.5x; last 2s earn 0.8x.
14. Lua script atomically updates `room:{id}:leaderboard` sorted set on Redis. Returns updated full rankings.
15. Score change triggers `LeaderboardUpdate GameEvent` broadcast to all Flutter clients.

2.4 Match End & Persistence Phase

16. After all 5 rounds complete, room service publishes `match.finished` event to RabbitMQ. Sends `MatchEnd GameEvent` to Flutter clients.
17. Persistence Worker consumes `match.finished`. Upserts `match_history` document using `$setOnInsert` to prevent double-writes on retry.
18. Bulk write updates `users.matchesPlayed` and `users.wins` for all participants.
19. Room Redis keys expire via TTL (30 min). No manual DEL required.
20. Analytics Worker consumes `match.finished` from `match-analytics-queue` (separate fan-out) for async pipeline processing.

3. Data Models

3.1 MongoDB Collections

Collection	Fields	Indexes
users	_id, username, rating, matchesPlayed, wins	email (unique), (rating DESC) for leaderboard queries
questions	_id, text, options[], correctIndex, difficulty, topic, avgResponseTimeMs	(difficulty, topic) compound for balanced question selection
match_history	_id, roomId, players[{userId, username, finalScore, rank, answersCorrect, avgResponseTimeMs}], rounds, winner, createdAt, duration	(players.userId, createdAt DESC) for per-user match history

users

```
{ _id, username, rating, matchesPlayed, wins }
```

questions

```
{ _id, text, options: [string], correctIndex, difficulty: "easy"|"medium"|"hard",  
  topic, avgResponseTimeMs }
```

match_history

```
{ _id, roomId, players: [{ userId, username, finalScore, rank, answersCorrect,  
  avgResponseTimeMs }], rounds, winner, createdAt, duration }
```

3.2 Redis Data Model

Key	Type	TTL	Purpose
matchmaking:pool	Sorted Set	—	Waiting players sorted by rating
room:{id}:players	Hash (userId → JSON)	30 min	Room membership & player metadata
room:{id}:state	String (JSON)	30 min	Room state machine value
room:{id}:round	String	30 min	Current active round index
room:{id}:questions	List	30 min	Ordered question IDs for the match
room: {id}:leaderboard	Sorted Set	30 min	Live score rankings
room:{id}:answers: {round}	Hash (userId → JSON)	30 min	Per-round submitted answers
room:lock:{id}	String (mutex)	10 s	Distributed lock for room creation

Note: All room:{id}:* keys carry a 30-minute TTL set at creation time. The TTL is refreshed on every write. room:lock:{id} carries a 10-second TTL as a safety release for abandoned locks.

3.3 Lua Script — Atomic Leaderboard Update

All leaderboard mutations use this Lua script to eliminate race conditions. The script executes atomically on the Redis server — no two goroutines can interleave their reads and writes:

```
local current = redis.call('ZSCORE', KEYS[1], ARGV[1]) if not current then current = 0
end redis.call('ZADD', KEYS[1], current + ARGV[2], ARGV[1]) return
redis.call('ZRANGE', KEYS[1], 0, -1, 'WITHSCORES')
```

4. gRPC Service Definitions

All three Go services expose gRPC interfaces. The Flutter client consumes the streaming RPCs via the generated Dart stubs. The .proto file is the single source of truth — generated stubs are committed to the repo before any business logic is written.

4.1 RPC Methods Reference

Service	RPC method	Type	Purpose
MatchmakingService	JoinMatchmaking	Unary	Add player to Redis waiting pool
MatchmakingService	LeaveMatchmaking	Unary	Remove player from pool
MatchmakingService	SubscribeToMatch	Server stream	Push match events to Flutter client
QuizService	GetRoomQuestions	Unary	Fetch ordered question list
QuizService	SubmitAnswer	Unary	Record player answer, return ack
QuizService	StreamGameEvents	Server stream	Broadcast questions, scores, timers
ScoringService	CalculateScore	Unary	Compute score for a single answer
ScoringService	GetLeaderboard	Unary	Return current sorted leaderboard

4.2 GameEvent Oneof Union

All live events from the server to Flutter are multiplexed through a single GameEvent message using a protobuf oneof field. This allows Flutter to handle all event types in a single stream handler:

```
message GameEvent {
  oneof event {
    LeaderboardUpdate leaderboard = 2;
    MatchEnd          match_end   = 4;
    TimerSync         timer_sync  = 6;
    QuestionBroadcast question    = 1;
    RoundResult        round_result = 3;
    PlayerJoined        player_joined = 5;
  }
}
```

5. RabbitMQ Topology

All events flow through a single topic exchange named `sx`. Consumers are independent goroutines that can scale horizontally. The `answer-processing-dlq` catches messages that fail after 3 delivery attempts — all DLQ arrivals are logged with the full message payload for audit and anti-cheat review.

Exc h.	Routing key	Queue	Consumer	Purpose
sx	match.created	match-created-queue	Room Service	Notifies players when matched
sx	answer.submitted	answer-processing-queue	Scoring Worker	Triggers score calculation
sx	round.completed	round-completed-queue	Quiz Engine	Advances to next round
sx	match.finished	match-finished-queue	Persistence Worker	Writes <code>match_history</code> to MongoDB
sx	match.finished	match-analytics-queue	Analytics Worker	Fan-out to analytics pipeline
—	—	answer-processing-dlq	Dead Letter Queue	Catches failed answer messages

5.1 Dead Letter Queue Strategy

- Max retry count: 3 per message
- On 3rd failure: message routes to `answer-processing-dlq` with full payload intact
- DLQ messages are logged to structured logger with `roomId`, `userId`, `round`, `timestamp`
- Manual replay available via admin script for legitimate failures
- Suspicious patterns (e.g. identical answer payloads replayed 100+ times) flagged for anti-cheat review

5.2 Idempotent Consumer Pattern

Every consumer that writes state must check for duplicate processing before doing any work. The pattern is:

21. Extract deduplication key: (`roomId`, `round`, `userId`) for answers; (`roomId`) for match lifecycle events.
22. Check: `HEXISTS room:{id}:answers:{round} {userId}` — if true, ACK the message and return without processing.
23. Process and write state.
24. ACK the message. This makes DLQ retries unconditionally safe.

6. Flutter Client Architecture

6.1 Screens & State Management

Screen	Riverpod provider	Key elements
Matchmaking	StreamProvider (SubscribeToMatch)	AnimationController (pulsing ring), username input, player count display, cancel button, ETA label
Quiz Gameplay	StreamProvider (StreamGameEvents)	Question card, 4 option buttons, countdown ring (CustomPainter), mini leaderboard overlay, optimistic answer color flash
Full Leaderboard	StateProvider (leaderboard snapshot)	Ranked list, gold/silver/bronze badges for top 3, animated position arrows (AnimatedList)
Results	FutureProvider (match end payload)	Winner display, personal stats (correct answers, avg response time), XP earned, rematch button

6.2 gRPC Stream Lifecycle

The StreamProvider for StreamGameEvents is scoped to the match router widget. It is created when the user enters a match and automatically disposed when they leave the results screen. This prevents orphaned gRPC connections and memory leaks.

Stream event handling pattern in Flutter:

- QuestionBroadcast → update question text, options, and reset AnimationController with deadline duration
- LeaderboardUpdate → update scores map, compute rank delta, trigger AnimatedList slide transitions
- RoundResult → highlight correct/wrong answers for 1.5s, then advance state
- MatchEnd → navigate to ResultsScreen, pass full scores payload
- PlayerJoined → show brief toast notification
- TimerSync → recalibrate AnimationController if drift > 500ms

6.3 Reconnection Logic

25. On stream error, show reconnecting banner in UI. Buffer last known leaderboard state in local Riverpod state.
26. Retry SubscribeToMatch with exponential backoff: 500ms, 1s, 2s. Include last received sequence_number in request.
27. Server replays any missed events from in-memory ring buffer (last 20 events per room).
28. Flutter snaps back to live game state. User sees minimal disruption.
29. After 3 failed retries, show match abandoned screen and return to lobby.

6.4 Optimistic UI

When a player taps an answer, Flutter immediately highlights the selected option before the server confirms correctness. The answer button state transitions are:

- Idle → Selected (local, immediate on tap)
- Selected → Correct (server RoundResult confirms) — green flash, brief scale animation
- Selected → Wrong (server RoundResult disconfirms) — red flash
- All buttons disabled after first tap in the same round

7. Real Engineering Challenges & Solutions

These are the race conditions, failure modes, and edge cases that must be explicitly handled. Each one has a concrete resolution — vague solutions like 'add retry logic' are insufficient.

Scenario	Trigger	Resolution
Double round completion	Two players submit last answer simultaneously	SETNX room:{id}:round:{n}:closed 1 EX 30 — only one goroutine wins the set; others return immediately
Timer desync	Server vs client clock drift	Server broadcasts absolute Unix deadline timestamp — Flutter computes deadline - now() locally rather than counting down a duration
Zombie rooms	Player disconnects without leaving	Heartbeat ping every 5s; if no ack in 15s mark player disconnected in room hash, broadcast PlayerLeft event, continue match
Worker crash mid-scoring	Score partially written before crash	Idempotent scoring: check HEXISTS room:{id}:answers:{round} {userId} before writing; DLQ retries are safe
Leaderboard race condition	Concurrent score updates clobber each other	Redis Lua script runs atomically on server: ZSCORE → add delta → ZADD in a single script call
Late joiner	Player enters during round 1	Room service returns state + current question + leaderboard snapshot as first event on stream — client shows loading state then snaps to live
Network partition	gRPC stream drops mid-match	Client holds last known sequence_number; on reconnect sends it in SubscribeToMatch; server replays from in-memory ring buffer (20 events)

7.1 Double Round Completion — Detailed

When the last player submits their answer at the same time as the server timer fires, two goroutines may simultaneously attempt to close the round. Without a guard, round_end is broadcast twice and scoring runs twice.

Resolution: before processing a round close, each goroutine attempts:

```
SETNX room:{id}:round:{n}:closed 1 EX 30
```

Only the goroutine that successfully sets this key (returns 1) proceeds to broadcast round_end and advance state. All others return immediately. The 30s TTL ensures the key expires even if the goroutine crashes after setting it.

7.2 Zombie Rooms — Heartbeat Design

Each Flutter client sends a heartbeat acknowledgment every 5 seconds in response to TimerSync events. Server-side per-player goroutine tracks last heartbeat:

- If no heartbeat in 15s: mark player as disconnected in room:{id}:players hash
- Broadcast PlayerLeft GameEvent to remaining players
- If only one player remains: wait 60s for reconnection before ending match
- If zero players remain: publish match.finished immediately (match_history records 0 players)

8. Solo Developer Strategy

All technical requirements in this document — three Go services, gRPC with server streaming, RabbitMQ, Redis, MongoDB, and the full Flutter client — are preserved. What changes is the execution strategy. A single developer achieves this in one day by treating AI code generation as a force-multiplier for every piece of infrastructure boilerplate, working in a strict dependency order, and keeping all three services in a monorepo so there is zero context-switching overhead.

8.1 Core Principles for Solo Execution

- Generate, don't type. The .proto file, Lua scripts, RabbitMQ consumer scaffolding, MongoDB seed data, and Flutter widget tests are all generated via Claude. The developer writes only business logic.
- Single Docker Compose file. One command brings up Redis, RabbitMQ, and MongoDB with health checks and correct dependency ordering. Never start an individual service manually.
- Monorepo layout. All three Go services live under /services/matchmaking, /services/quiz, /services/scoring. A shared /pkg/keys package defines all Redis key constants and is imported by all three. Key names never drift.
- Backend-first, grpcurl-verified. Complete and smoke-test the entire Go backend with grpcurl before writing a single line of Flutter. This isolates bugs to one layer at a time.
- Mock gRPC server for Flutter. While the backend is being built, Flutter compiles against a local mock server (generated stub server with hardcoded responses). Switch to the real server once both layers are stable.
- Seed MongoDB from JSON. Use Claude to generate a questions.json file with 30 balanced questions (12 easy, 15 medium, 3 hard). Import with mongoimport in one command — never write questions by hand.

8.2 Monorepo Structure

```
quiz-battle/  docker-compose.yml  proto/      quiz.proto      (single source
of truth)    services/      matchmaking/main.go  quiz/main.go    scoring/main.go
pkg/         keys/keys.go    (all Redis key constants)  models/models.go
(shared structs)  flutter/      lib/         services/quiz_service.dart
providers/game_state.dart  screens/     seed/        questions.json
```

9. One-Day Execution Plan

The day runs approximately 13 hours. Every block below has a hard stop time. If a block runs over, skip polish within it and move on — integration is more important than any single component being perfect. All requirements remain in scope; the pace is managed by AI-assisted generation, not by cutting features.

Time block	Focus	Exit criteria
07:00 – 08:00	Environment	docker compose up healthy; protoc generates Go + Dart stubs; monorepo compiles clean
08:00 – 09:30	Shared layer	keys.go complete; Redis client wrappers written; Lua leaderboard script tested with redis-cli
09:30 – 11:00	Matchmaking svc	JoinMatchmaking, room creation with lock, match.created published; verified with grpcurl
11:00 – 12:30	Quiz engine svc	Question selection, startRound timer, StreamGameEvents broadcasting; verified with grpcurl
12:30 – 13:00	Lunch break	Full stop — mandatory cognitive reset
13:00 – 14:30	Scoring svc	CalculateScore, idempotent worker, Lua leaderboard update, DLQ configured; end-to-end backend verified
14:30 – 15:00	Persistence	Persistence worker consuming match.finished, match_history upserted to MongoDB; mongoimport seed complete
15:00 – 16:30	Flutter backend	QuizService, Riverpod providers, gRPC channel; matchmaking screen connects to real backend
16:30 – 17:30	Flutter gameplay	Gameplay screen: question card, countdown ring, optimistic answer coloring, mini leaderboard
17:30 – 18:30	Flutter results	Full leaderboard + results screen; reconnection logic wired; all 4 screens navigable end-to-end
18:30 – 19:30	Integration	Two emulator instances complete a full match: join → 5 rounds → results → match_history in MongoDB
19:30 – 20:00	Race condition audit	All 7 checklist items verified (see section 9.9)

9.1 Hour 07:00–08:00 — Environment & Proto

This block is the critical path. Nothing else can start until it is done.

30. Use Claude to generate docker-compose.yml with redis:7, rabbitmq:3-management, and mongo:6. Include healthcheck for each service and depends_on with condition: service_healthy. Run docker compose up -d.
31. Use Claude to generate the complete quiz.proto file: all three services, all message types, the GameEvent oneof union with all 6 variants. This is the single source of truth — review it carefully before running protoc.

32. Run protoc for Go stubs: `protoc --go_out=. --go-grpc_out=. proto/quiz.proto`
33. Run protoc-dart for Flutter stubs: `protoc --dart_out=grpc:flutter/lib/proto proto/quiz.proto`
34. Verify both compile: `go build ./...` and `flutter pub get`. Fix any import issues before proceeding.

9.2 Hour 08:00–09:30 — Shared Layer

Write `pkg/keys/keys.go` first — this is imported by all three services and is the single place where Redis key names are defined. Any key name typo across services will cause silent failures that are very hard to debug.

```
const (
    MatchmakingPool    = "matchmaking:pool"
    RoomPlayers        = "room:%s:players"
    RoomState          = "room:%s:state"
    RoomRound          = "room:%s:round"
    RoomQuestions      = "room:%s:questions"
    RoomLeaderboard    = "room:%s:leaderboard"
    RoomAnswers        = "room:%s:answers:%d"
    RoomLock            = "room:lock:%s"
    RoomRoundClosed    = "room:%s:round:%d:closed"
)
```

Use Claude to generate the Redis client wrapper functions (one function per key operation). Use Claude to generate the Lua leaderboard script and test it with `redis-cli EVAL` before wiring it into Go.

Also in this block: use Claude to generate `questions.json` with 30 questions (12 easy, 15 medium, 3 hard, spread across 5 topics). Run `mongoimport --db quizbattle --collection questions --file seed/questions.json`. Verify count with `db.questions.countDocuments()`.

9.3 Hour 09:30–11:00 — Matchmaking Service

Implement `services/matchmaking/main.go`. Wire the generated gRPC stub server. Focus in this order:

35. `JoinMatchmaking`: ZSCORE check for duplicate, then ZADD to `matchmaking:pool` with `score=rating`.
36. `LeaveMatchmaking`: ZREM from pool.
37. Background poller goroutine (`time.NewTicker 500ms`): ZCARD check → attempt room creation.
38. Room creation: SET NX `room:lock:{id}` with 10s TTL → MULTI/EXEC writing all four room keys → publish `match.created` to RabbitMQ.
39. `SubscribeToMatch`: open a server-streaming channel per player, stored in a `sync.Map` keyed by `userId`. Room events write to these channels.
40. Smoke test with `grpcurl`: `grpcurl -plaintext -d '{"userId":"u1","rating":1200}' localhost:50051 MatchmakingService/JoinMatchmaking`

Note: Heartbeat tracking: store last heartbeat timestamp per player in a `sync.Map`. A separate goroutine checks every 5s — if stale by >15s, mark disconnected and broadcast `PlayerLeft`. Wire this up now, not later.

9.4 Hour 11:00–12:30 — Quiz Engine Service

Implement `services/quiz/main.go`. This service is the orchestrator of the round loop.

41. RabbitMQ consumer for `match.created`: on consume, call `selectQuestions()` and write to room: `{id}:questions` list.
42. `selectQuestions()`: query MongoDB with `{ difficulty: { $in: [...] } }` using weighted selection — 30% easy, 50% medium, 20% hard. No repeat topic in adjacent rounds.
43. `startRound()`: broadcast `QuestionBroadcast GameEvent` with absolute Unix deadline (`time.Now().Add(15*time.Second).Unix()`). Start `time.AfterFunc(15*time.Second, closeRound)`.
44. `closeRound()`: SETNX room:`{id}:round:{n}:closed` 1 EX 30 guard. Broadcast `RoundResult`. If more rounds remain, call `startRound()` after a 2s pause. Otherwise publish `match.finished`.
45. `SubmitAnswer`: publish `answer.submitted` event to RabbitMQ. Return `AnswerAck` immediately — do not wait for scoring.
46. Smoke test: run matchmaking + quiz services together and trace a full 5-round game in the terminal logs using `grpcurl`.

9.5 Hour 13:00–14:30 — Scoring Service & Workers

Implement `services/scoring/main.go`. This is the most concurrency-sensitive service — be deliberate.

47. `CalculateScore` gRPC: `basePoints = 100 * (1 if correct, 0 if wrong)`. `speedMultiplier`: `answerTimeMs < 5000 → 1.5`, `answerTimeMs > 13000 → 0.8`, else 1.0.
48. Use Claude to generate the RabbitMQ consumer for `answer-processing-queue` with idempotency check and DLQ routing. Review the generated code for the dedup key pattern before using it.
49. Wire the Lua leaderboard script from `pkg/keys` into the scoring pipeline. After each update, retrieve full rankings and publish `leaderboard.updated` to RabbitMQ.
50. Configure `answer-processing-dlq`: set `x-dead-letter-exchange` and `x-max-delivery-count=3` on the `answer-processing-queue` declaration.
51. `GetLeaderboard` gRPC: `ZREVRANGE room:{id}:leaderboard 0 -1 WITHSCORES`.

9.6 Hour 14:30–15:00 — Persistence Worker

The persistence worker is a separate goroutine within `services/scoring/main.go` (not a fourth service). It consumes `match.finished` from `match-finished-queue`.

52. Consume `match.finished`: build `match_history` document from room:`{id}:leaderboard` and room metadata.
53. Upsert with filter `{ roomId }` and update `{ $setOnInsert: document }` to prevent double-writes.
54. Bulk write to `users` collection: increment `matchesPlayed` for all players, increment `wins` for rank-1 player.
55. Verify via mongosh: `db.match_history.find().pretty()` should show the complete match document.

Note: Analytics worker: also consuming match.finished from match-analytics-queue. For this build, the consumer just logs the payload to structured output — the actual analytics pipeline is a stub. The queue binding is real; the processing is deferred.

9.7 Hour 15:00–16:30 — Flutter Backend Layer

The backend is now fully functional and verified with grpcurl. Switch the Flutter mock server to the real gRPC address.

56. flutter create quiz_battle. Add pubspec dependencies: grpc, protobuf, riverpod, flutter_riverpod, collection.
57. Copy generated Dart stubs into lib/proto/. Verify flutter pub get compiles.
58. QuizService class: singleton gRPC ClientChannel on localhost:50051. Methods: joinMatchmaking(), subscribeToMatch(), streamGameEvents(), submitAnswer().
59. GameState provider (StateNotifier): holds currentScreen, currentQuestion, scores Map, round, timer deadline.
60. StreamProvider wrapping subscribeToMatch(): auto-disposes on navigation away. Emits MatchEvent sealed class variants.
61. Matchmaking screen: TextField for username, Start Battle button. On tap, call joinMatchmaking() and show pulsing AnimationController ring. On match_found event, navigate to gameplay screen.

9.8 Hour 16:30–18:30 — Flutter UI Screens

Gameplay screen (16:30–17:30)

62. On QuestionBroadcast: update question text + 4 option buttons. Reset AnimationController duration to (deadlineUnix - now) seconds.
63. Countdown ring: CustomPainter drawing an arc from 2π to 0 driven by AnimationController.value. Stroke color transitions amber → red in final 3s.
64. Option buttons: on tap, call submitAnswer() and immediately set local selectedIndex state. On RoundResult event, color selectedIndex green or red for 1.5s, then reset.
65. Mini leaderboard overlay at bottom: 2-row display of player names and scores, updated on each LeaderboardUpdate event.

Full leaderboard + results screens (17:30–18:30)

66. Full leaderboard screen: AnimatedList of ranked rows. Each row shows position, username, score. Gold/silver/bronze background for positions 1–3. On LeaderboardUpdate, compute rank delta per player and show up/down arrows.
67. Results screen: on MatchEnd event, navigate here. Show winner name prominently, personal finalScore + rank + answersCorrect + avgResponseTimeMs. Play Again → call LeaveMatchmaking then navigate to lobby.
68. Reconnection banner: wrap StreamProvider in a ref.listen that detects error state → show persistent top banner 'Reconnecting...' with exponential backoff retry (500ms, 1s, 2s). On reconnect, pass last sequence_number.

9.9 Hour 18:30–19:30 — Integration & End-to-End Test

Run two Flutter instances (Android emulator + iOS simulator, or emulator + browser) against the local Go services. Walk through a complete match from both perspectives simultaneously.

End-to-end test checklist:

- Both clients join matchmaking → both receive `match_found` within 30s
- Question appears on both screens simultaneously with identical deadline timestamp
- Player 1 submits answer → mini leaderboard updates on both screens within 200ms
- Round timer fires → both screens show correct answer highlight, then advance
- All 5 rounds complete → `MatchEnd` received → both navigate to results
- `match_history` document visible in MongoDB with both players' final scores
- Disconnect test: kill Flutter instance mid-game → reconnecting banner appears on restart → state recovered

9.10 Hour 19:30–20:00 — Race Condition Audit

Verify each of the 7 engineering challenges from section 7 is concretely handled in the running code:

- Double round completion — search codebase for `SETNX room:{id}:round:{n}:closed` — must exist in `closeRound()`
- Timer desync — verify `QuestionBroadcast` payload contains `deadlineUnix` (absolute), not a duration field
- Zombie rooms — verify heartbeat goroutine is started per player in `SubscribeToMatch` handler
- Worker crash mid-scoring — verify `HEXISTS` check is the first line of the scoring consumer handler
- Leaderboard race — verify Lua script path is taken for all `ZADD` calls; no raw `ZADD` anywhere in scoring service
- Late joiner — verify `SubscribeToMatch` sends state + current question snapshot as first event for mid-match joins
- Network partition — verify `sequence_number` is included in `SubscribeToMatch` request and server ring buffer replay is wired

10. AI-Assisted Development — Detailed Prompts

Every boilerplate-heavy task below should be delegated to Claude. The developer reviews and integrates the output — not types it. These prompts are ordered to match the execution plan in section 9.

Task (order matches section 9)	Specific prompt to use
Docker Compose (09:00)	Generate a docker-compose.yml for redis:7, rabbitmq:3-management, and mongo:6 with healthcheck on each, depends_on with service_healthy, and named volumes. Include a Go app service stub.
.proto file (09:00)	Generate a complete quiz.proto for a real-time multiplayer quiz system with MatchmakingService (JoinMatchmaking, LeaveMatchmaking, SubscribeToMatch server-stream), QuizService (GetRoomQuestions, SubmitAnswer, StreamGameEvents server-stream), ScoringService (CalculateScore, GetLeaderboard). Include all message types and a GameEvent oneof with QuestionBroadcast, LeaderboardUpdate, RoundResult, MatchEnd, PlayerJoined, TimerSync.
MongoDB seed questions (08:30)	Generate a questions.json array of 30 quiz questions: 12 easy, 15 medium, 3 hard. Spread across 5 topics: science, history, geography, sports, tech. Each object must have: text (string), options (4-element array), correctIndex (0-3), difficulty, topic, avgResponseTimeMs.
Redis Lua script (08:30)	Write a Redis Lua script that atomically reads the current score for ARGV[1] from sorted set KEYS[1], adds ARGV[2] to it, writes it back with ZADD, and returns the full ZRANGE WITHSCORES. Explain in comments why this is race-condition-free.
RabbitMQ consumer + DLQ (13:00)	Write a Go RabbitMQ consumer for a queue named answer-processing-queue. It must: (1) check a Redis HEXISTS key for idempotency before processing, (2) call a provided processAnswer(msg) function, (3) ACK on success, (4) NACK without requeue on the 3rd delivery (x-death header count >= 3) to route to DLQ, (5) NACK with requeue for transient errors.
Concurrent flow analysis (19:30)	Here are two Go goroutine code paths that both call closeRound() for the same room. Identify all possible interleavings and explain whether the SETNX guard at the start of closeRound() is sufficient to prevent double round completion.
Flutter widget tests (after build)	Write Riverpod widget tests for the GameplayScreen that simulate: (1) receiving a QuestionBroadcast stream event and verifying the question text renders, (2) tapping an option and verifying submitAnswer is called and buttons are disabled, (3) receiving a RoundResult event and verifying the correct option turns green.